# 電腦抽象、ISA 與可量化效能

從一個 `sum += a[i]` 到電晶體切換，中間隔著多層契約。本章的目標是讓每一層都有可量化的邊界。

本課以 MIPS32 作為教學 ISA。MIPS 是本書選用的指令集範例，不代表所有現代處理器都直接執行 MIPS。ARM 與 x86 會在需要比較時出現。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 前置知識與學習成果

必備工具包括二進位、科學記號、比率與加權平均。課程結束時，學生應能以單位完整的式子預測程式執行時間，並說明預測在哪些條件下失效。

本課所稱「電腦效能」必須附上工作負載。沒有工作負載的「處理器很快」是測量條件不完整的句子。

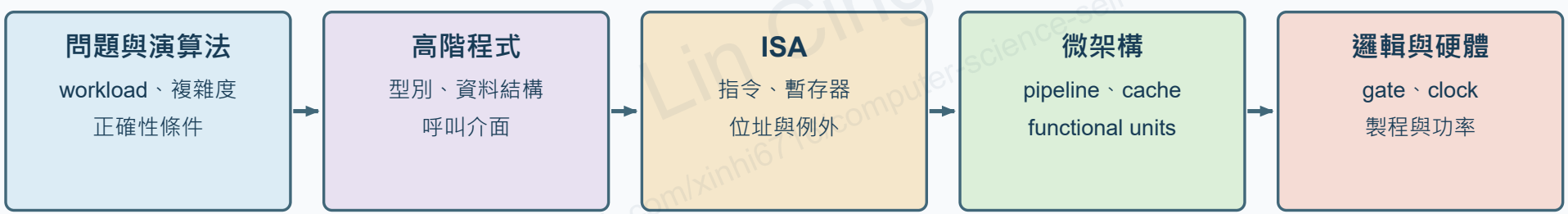
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.1 電腦抽象層

電腦系統用分層管理複雜度。應用軟體使用系統呼叫與函式庫，編譯器產生 ISA 指令，微架構把指令展開為資料路徑操作，邏輯與實體層實現暫存器、加法器與記憶體。

相鄰層透過介面交換可觀察行為。介面讓上層不必知道下層每個電晶體，但最佳化會跨層傳遞資訊，例如資料局部性影響編譯器排程與快取命中。

### 抽象堆疊以介面承接上下層契約




上層提出可觀察契約，下層提供實作；最佳化資訊可跨層傳遞，但不可破壞介面。

抽象不是隱藏一切。軟體可透過指令、位址、例外與效能計數器觀察下層的部分行為。

# ISA 與 microarchitecture

Instruction set architecture，中文為指令集架構，定義軟體可見的指令、暫存器、資料型別、定址、權限與例外。兩臺處理器可以有相同 ISA，卻有不同 pipeline 深度、cache 大小與執行單元。

Microarchitecture，中文為微架構，定義如何實作 ISA。它決定某條指令在各時鐘週期走到哪個階段，以及處理器能否同時維持多個 in-flight instructions。

若程式結果與 ISA 契約一致，微架構可以自由重排、預測與快取。若重排讓程式觀察到不同記憶體結果，必須由同步規則約束。

ISA 像函式介面，微架構像函式內部的資料結構與演算法。API 相同，成本與並行度可以不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 應用類別與設計取捨

個人電腦、伺服器、嵌入式與 mobile 系統有不同的限制。伺服器最佳化總吞吐量與 availability，mobile 系統受到 energy 與 thermal envelope 限制，嵌入式控制則重視 worst-case response。

同一項技術在不同市場的價值不同。有大容量 cache 可改善一般負載，但會占面積與增加 static power。低功耗休眠可延長電池壽命，卻可能提高喚醒延遲。

設計取捨至少包含效能、成本、功率、可靠度與可程式性五個量，不能只報告其中一項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 課程地圖

第 2 章建立 ISA，第 3 章處理整數與浮點算術，第 4 章把 ISA 變成 datapath 與 pipeline。第 5 章處理 cache 與 virtual memory，第 6 章處理 storage 與 I/O，第 7 章討論 multicore、multiprocessor 與 cluster。

後面的章節反覆使用本章三個工具：單位分析、局部化比較、以 workload 驗證。看到新的架構名詞時，先問它改變了哪個資源與哪個時間分量。

| 主線 | 本章建立的模型 | 後續機制 |
| --- | --- | --- |
| 資料 | operands、結果與 IC | 第2章暫存器/位址 → 第3章算術 → 第4章 datapath → 第5章 cache |
| 控制 | ISA 的可觀察狀態與時間 | 第2章 branch/call → 第4章 hazard/exception → 第6章 I/O completion |
| 通訊 | 搬移 bytes 與共享資源 | 第5章 coherence → 第6章 DMA/links → 第7章 synchronization/networks |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 1.2 程式如何落到硬體

高階語言先經 compiler 產生 assembly，assembler 再產生 machine instructions。Static linker 合併 object files，loader 建立 process 可用的位址空間，處理器最後依 ISA 解碼並執行。

每一步都產生可檢查內容。編譯器可改變指令數與暫存器使用；assembler 決定 encoding；linker 解決符號與 relocation；loader 建立初始記憶體映射。

效能分析不能只比較 C 原始碼行數，因為最佳化層級、資料型別與編譯器輸出會改變 instruction count。

請區分 source code、assembly 與 machine code。本課的 ISA 圖都畫在 assembly 與機器碼之間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 抽象介面與可攜性

ISA 讓同一個 executable 能在實作相同 ISA 的處理器上執行。作業系統還要提供 system call 與 binary interface，程式才完整可攜。

應用 binary interface，簡稱 ABI，規定暫存器保存方式、呼叫慣例、資料對齊與 executable 格式。它比 ISA 更接近程式工具鏈。

一個處理器可能維持舊 ISA 以執行既有程式，同時在內部把舊指令解譯成新的 micro-operations。這是實作選擇，不表示 ISA 消失。

區分 compatibility、portability 與 equivalence。能執行不代表得到相同效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.3 開啟電腦外殼

系統由 processor、volatile memory、nonvolatile storage、I/O 與 interconnect 構成。輸入裝置把外部狀態轉成資料，輸出裝置把資料轉成可觀察效果。

處理器內有 datapath 與 control，外部有 DRAM 與 persistent storage。匯流排或點對點 interconnect 傳遞位址、資料與控制訊息。

周邊裝置不是「電腦之外」的附屬品。鍵盤、滑鼠、顯示器與網路都會透過 interrupt、DMA 或 memory-mapped I/O 影響系統效能。

以下固定 USB 滑鼠、host-controller DMA ring 與一個等待輸入的應用程式。圖中的 buffer 是不同責任邊界，不假設每個箭頭都複製整份資料。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

### 一筆 USB 滑鼠事件：資料、buffer 與通知




完成前 ring entry 由 controller 使用；應用程式必須等排程後才能處理事件。

USB report → DMA ring → input queue → application event queue

Controller 先將 report 寫入 driver 配置的 ring，完成通知讓 driver 取得已完成項，再交給 input queue；scheduler 喚醒應用後，應用才取走事件。DMA 完成前不能回收 ring slot；應用醒來時間也不等於封包抵達時間。

### 從滑鼠看周邊介面

滑鼠感測器產生位移資訊。裝置控制器把它整理成封包，透過 USB 或 Bluetooth 等介面傳給 host controller。驅動程式與作業系統再交給 input subsystem。

每次移動不一定立即喚醒應用程式。系統會批次處理事件以降低 CPU 喚醒成本，這形成 latency 與 energy 的取捨。

測量「滑鼠反應慢」時要分清楚裝置取樣率、匯流排傳輸、作業系統排程、應用程式 frame rate 與顯示器更新率。

這個例子用來練習端到端延遲分解，數字未指定時不可假設由單一元件的速度決定。

### 從平板看系統組成

Tablet 使用 system on chip，簡稱 SoC，把多個 CPU cores、GPU、media processor、memory controller 與 I/O controller 放在同一晶片。整合降低封裝與通訊成本，也提高 power density 與 thermal 管理難度。

使用者感受的 response time 包含 touch sampling、GUI composition、computation 與 display scan-out。處理器 burst 很快完成時，仍需等待顯示 pipeline。

Mobile workload 常見短暫 burst 與長時間 idle。平均值會掩蓋 tail latency 與 energy state transition。

### 主機板與資料路徑

主機板提供芯片與周邊的實體連接。現代系統多採 point-to-point link，不再把所有裝置接在同一條共享 bus，因為共享 bus 的 contention 與電氣負載會限制頻寬。

晶片內的 interconnect 連接 cores、last-level cache 與 memory controller。Cache coherence 維持共享資料的可觀察一致性。

拓樸決定 traffic hop count 與 contention。頻寬峰值不能直接當成應用程式可得到的持續 bandwidth。

### 資料保存層級

Register 與 cache 屬 volatile storage，斷電後資料消失。DRAM 也是 volatile。Flash、magnetic disk 與 optical media 屬 nonvolatile storage。

可靠性與備份是不同問題。RAID 可容忍某些裝置故障，但無法取代異地備份。檔案系統 crash consistency 也不能只靠平均值中的 MTBF 保證。

Temporary copy 必須和 authoritative copy 區分。Cache line 與 write-back dirty bit 改變資料在哪裡最新，但不改變程式的 logical data model。

### 網路為何屬於系統組織

現代電腦的效能與可用性受 network latency、bandwidth 與 protocol overhead 影響。機架內與跨資料中心的距離不同，光速、交換器佇列與 congestion control 共同決定端到端時間。

資料可能先從 storage 進入 memory，再由 DMA engine 送到 network interface。CPU 不必逐 byte 搬移，但仍需設定 descriptor 與處理完成事件。

系統設計需一起考慮 computation、storage 與 network，否則算得再快的服務可能等待 I/O。

在無重疊、連線已建立的教學模型，固定 request encoding 0.2 ms、等待 0.3 ms、去程傳輸與傳播 2.0 ms、server computation 1.5 ms、回程 2.0 ms。總 latency 為 0.2+0.3+2.0+1.5+2.0=6.0 ms。只把 server computation 加速3倍，總時間為5.0 ms，整體 speedup=6/5=1.2。各數字是明定模型輸入，若網路傳輸與計算重疊，必須改以 dependency critical path 計算。

### 製程與成本

IC 技術以 transistor integration 與 feature size 描述。較小元件通常降低每單位功能的面積與動態能量，但 leakage、變異與製造成本會使成本下降不再自動持續。

晶圓良率使每顆可用晶片成本不是裸晶面積的線性函數。大晶片能整合更多功能，也承受較高 defective die 比率。

成本要指定範圍。單顆晶片製造、封裝、電源、冷卻、伺服器機架與營運 energy 的成本不同。

不要把「電晶體更多」當成「每個程式更快」。這個結論需要 parallel 或 architectural 利用。

## Die 面積、良率與成本

固定wafer diameter $D = 200$ mm、每片成本1000成本單位、每die面積 $A$（mm²）。邊緣損失的幾何近似採

$$N_{\text{die}} = \left\lfloor \frac{\pi (D/2)^2}{A} - \frac{\pi D}{\sqrt{2A}} \right\rfloor .$$

另明定負二項良率模型 $Y = (1 + dA_c/\alpha)^{-\alpha}$，$d = 0.5$ defects/cm²、clustering參數 $\alpha = 2$、$A_c = A/100$ cm²。此處100 mm²=1 cm²，不能直接把mm²代入defects/cm²。忽略封裝、測試、備援及其他成本，平均good-die成本為 $1000/(N_{\text{die}} Y)$。

| Die面積 | 幾何項與邊緣項 | Gross dies | Yield | 預期good dies | 每good die成本 |
|---|---|---|---|---|---|
| 100 mm² | floor(314.159−44.429) | 269 | (1+0.5/2)⁻²=0.64 | 172.16 | 5.8086 |
| 200 mm² | floor(157.080−31.416) | 125 | (1+1/2)⁻²=4/9 | 55.5556 | 18.0000 |

面積加倍同時降低gross dies及yield，本模型成本比為18/5.8086≈3.10，並非2。Good dies為期望值，實際產出為整數；本例的幾何/良率與成本數字皆為教學假設，非製造報價。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 1.4 效能的定義

Response time，中文為回應時間，是單一工作從開始到完成經過的時間。Throughput，中文為吞吐量，是單位時間完成的工作數。

若最佳化縮短 critical path，可能改善 response time。若增加流水線、batch 或多核心，可能主要改善 throughput。

任何延遲改善百分比都要用同一 baseline。$T_{\text{new}} = T_{\text{old}}/S$，其中 speedup $S > 0$。若時間減半，$S = 2$，時間節省比例是 $50\%$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 延遲與吞吐量

以輸送帶為例，單筆 work 經過五個 stations 的總時間是 latency。穩定狀態下每週期離開一筆，是 throughput。Pipelining 通常降低平均完成間隔，舊工作的 latency 未必等比例降低。

Batch processing 可攤薄 setup 成本，提高 throughput，但第一筆結果可能更晚。多核心能並行不同工作，卻受 shared resources 與 synchronization 限制。

公平比較要固定 workload、輸入大小、warm-up、軟體版本與量測視窗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 基準、比率與幾何平均

Benchmark 必須指定 workload、輸入大小、系統配置、compiler flags、warm-up 與量測視窗。這些條件不同時，兩個「SPEC 分數」不是同一個實驗。

以 reference machine 的程式 $i$ 執行時間 $T_{\mathrm{ref},i}$ 除以受測機器的 $T_{X,i}$，得到

$$r_{X,i} = \frac{T_{\mathrm{ref},i}}{T_{X,i}}.$$

分數愈大表示受測時間愈短。單一程式可以直接比較；多個程式若要形成摘要，通常取幾何平均

$$GM_X = \left( \prod_{i=1}^{n} r_{X,i} \right)^{1/n}.$$

幾何平均適合比率的原因可以直接證明。兩台機器 X、Y 的幾何平均比值為

$$\frac{GM_X}{GM_Y} = \left( \prod_{i=1}^{n} \frac{r_{X,i}}{r_{Y,i}} \right)^{1/n} = \left( \prod_{i=1}^{n} \frac{T_{Y,i}}{T_{X,i}} \right)^{1/n}.$$

# 基準、比率與幾何平均 (續)

每個程式的 $T_{\text{ref},i}$ 都在 X/Y 的比值中成對抵銷。因此 reference machine 換成另一台、各程式 reference time 各自不同時，X 對 Y 的幾何平均比值仍不變；算術平均沒有這個性質。

摘要分數仍不代表所有 workload。若某使用者的工作有 90% 時間落在單一 kernel，應以該 workload 重新加權或直接比較那支程式；不能只用幾何平均宣告普遍最快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## CPU time 的可重建模型

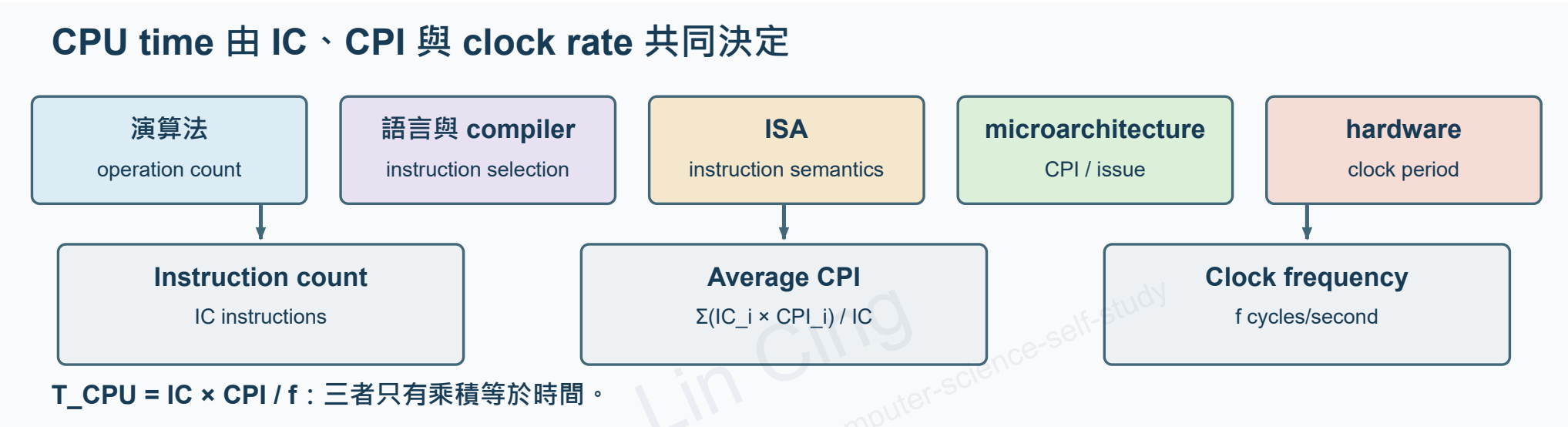
CPU time 計算CPU替程式執行的時間；以下不等式限定單執行緒、同一量測視窗，且user/system accounting不重複：

$$T_{\text{elapsed}} \geq T_{\text{CPU}} = T_{\text{user}} + T_{\text{system}}.$$

I/O等待、被排程器換出與OS替其他工作執行仍消耗elapsed time。多執行緒process若將各thread CPU time相加，總CPU-seconds可以超過elapsed seconds：4cores各工作1秒，總CPU time為4 CPU-s，elapsed為1 s。比較效能時必須明列thread/process accounting與使用的時間分母。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# CPU time 的可重建模型 （續）




先從 cycle 數開始：

$$C = IC \times CPI.$$

每個 cycle 長 $T_c$ 秒，所以

$$T_{\text{CPU}} = C\,T_c = IC \times CPI \times T_c = \frac{IC \times CPI}{f}.$$

# CPU time 的可重建模型 （續）

量綱是

$$\text{instructions} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}} = \text{seconds}.$$

## 完整比較例

同一程式在 A、B 上都執行 $I = 10^9$ 條 ISA instructions。A 的 CPI=2.0、clock period=250 ps；B 的 CPI=1.2、clock period=500 ps。

$$C_A = 10^9(2.0) = 2.0 \times 10^9, \qquad T_A = 2.0 \times 10^9(250\,\text{ps}) = 0.500\ \text{s}.$$

$$C_B = 10^9(1.2) = 1.2 \times 10^9, \qquad T_B = 1.2 \times 10^9(500\,\text{ps}) = 0.600\ \text{s}.$$

因此

$$S_{A/B} = \frac{T_B}{T_A} = \frac{0.600}{0.500} = 1.2.$$

A 比 B 快 1.2 倍。B 的 CPI 較低，但每個 cycle 較長，所以仍較慢。只比較 CPI、頻率或 instruction count 都會漏掉另外兩項。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# CPU time 的可重建模型 （續）

這個 $IC$ 相同的前提只在同一 ISA、同一程式與相同動態執行路徑成立。換 compiler、ISA 或輸入後，instruction count 可以直接改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 加權 CPI 與「少一條指令未必較快」

若類別 $i$ 有 $IC_i$ 條 instruction，平均每條需 $CPI_i$ cycles，總 cycles 與 CPI 是

$$C = \sum_i IC_i \, CPI_i, \qquad CPI_{\text{avg}} = \frac{\sum_i IC_i \, CPI_i}{\sum_i IC_i}.$$

分母必須是全部執行過的 instructions，不是類別數，也不是 source line 數。

### 完整反例

硬體有三類 instruction：

| 類別 | CPI |
|---|---|
| A | 1 |
| B | 2 |
| C | 3 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 加權 CPI 與「少一條指令未必較快」（續）

兩個 compiler 對同一高階語句產生：

| 序列 | A 條數 | B 條數 | C 條數 | 總 instruction |
|---|---|---|---|---|
| 1 | 2 | 1 | 2 | 5 |
| 2 | 4 | 1 | 1 | 6 |

序列 1：

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \qquad CPI_1 = 10/5 = 2.0.$$

序列 2：

$$C_2 = 4(1) + 1(2) + 1(3) = 9, \qquad CPI_2 = 9/6 = 1.5.$$

在相同 frequency 下，

$$T_1 = 10T_c, \quad T_2 = 9T_c, \quad S_{2/1} = 10/9 \approx 1.111.$$

序列 2 多一條 instruction，卻少一個 cycle。原因不是「instruction 較少一定快」，而是總 cycle 數才決定 CPU time。這個結果也提醒我們：instruction mix 改變時，不能把單一 CPI 當常數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 加權 CPI 與「少一條指令未必較快」（續）

量測時可用 profiler、ISA simulator 或 hardware performance counters 取得 $IC_i$ 與 cycle 來源；三種方法的覆蓋範圍與額外 overhead 必須另外記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 經典效能方程式的邊界

效能方程式是 accounting identity，不是保證任何局部改動會改善總時間。它假設 $IC$、CPI 與 $f$ 能被合理量測且 workload 固定。

CPU time 也不等於 wall-clock response time。若程式等待 I/O、作業系統排程或網路，必須把等待時間納入。

對多核平行程式，還需考慮使用 core 數 $p$、平行比例、負載不平衡與 synchronization overhead。第 7 章會用 Amdahl 與 Roofline 延伸本章。

若處理器頻率不變、CPI 降低，但 compiler 產生更多指令，總結果取決於兩個乘數的競賽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 完整效能預算例

甲程式執行 $2.0 \times 10^8$ 條 instructions，CPI=1.25，頻率 2.0 GHz。

1. cycles：$2.0 \times 10^8 \cdot 1.25 = 2.5 \times 10^8$。
2. cycle time：$1/(2.0 \times 10^9) = 0.5$ ns。
3. CPU time：$2.5 \times 10^8 \cdot 0.5\ \text{ns} = 0.125\ \text{s}$。

$$T_{\text{A}} = \frac{(2.0 \times 10^8)(1.25)}{2.0 \times 10^9} = 0.125\ \text{s}.$$

新 compiler 把 instruction count 降至 $1.6 \times 10^8$，但 CPI 升為 1.40，frequency 仍為 2.0 GHz：

$$C_{\text{B}} = 1.6 \times 10^8 (1.40) = 2.24 \times 10^8,$$

$$T_{\text{B}} = \frac{2.24 \times 10^8}{2.0 \times 10^9} = 0.112\ \text{s}, \qquad S = \frac{0.125}{0.112} = 1.1161.$$

Instruction count 降低 $20\%$，但 CPI 上升 $12\%$，所以時間只降低

$$(0.125 - 0.112)/0.125 = 10.4\%.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 完整效能預算例 （續）

如果把 B 的 CPI 誤用成 A 的 1.25，會得到 $T = 0.100$ s、speedup 1.25；這個答案忽略 instruction mix 已改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.5 功耗牆

Dynamic power 的簡化式是

$$P_{\text{dyn}} = CV^2 f\,\alpha,$$

其中 $C$ 為被切換電容，$V$ 為電壓，$f$ 為頻率，$\alpha$ 為 activity factor。提高頻率通常也要提高 voltage，因此 voltage-frequency scaling 會使 power 成長快於線性。

晶片還有 leakage power。降低供應電壓本身通常會降低漏電，但也會降低速度；若為了在低電壓維持速度而同時降低臨界電壓，次臨界漏電可能增加，製程變異的相對影響也可能變大。

散熱限制決定可持續的 power density。接近或超過平台的溫度、電流或功率限制時，時脈可能降低或部分資源停用；TDP是散熱設計參數，不是所有處理器固定且唯一的throttling門檻。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 單核到多核與 Amdahl 的完整推導

功率密度、clock distribution、可用的 instruction-level parallelism 與記憶體延遲共同使提高單核頻率不再是永續的效能來源。多核提高 aggregate throughput，並可能縮短能把工作切成獨立部分的大型問題 elapsed time。

令固定工作的 baseline time 為 $T_0$，其中 $fT_0$ 可由 $N$ 個 worker 分擔，$(1-f)T_0$ 必須序列執行。假設工作均分且沒有新增成本：

$$T_N = (1-f)T_0 + \frac{fT_0}{N} = T_0\left(1 - f + \frac{f}{N}\right).$$

$$S_N = \frac{T_0}{T_N} = \frac{1}{1 - f + f/N}.$$

當 $N \to \infty$，平行部分趨近 0，但序列部分仍在：

$$S_\infty = \frac{1}{1-f}.$$

### 反解所需平行比例

若要達到指定的 finite-$N$ speedup $S$，必須

$$\frac{1}{S} = 1 - f + \frac{f}{N} = 1 - f\left(1 - \frac{1}{N}\right),$$

因此

$$f \geq \frac{N(S-1)}{S(N-1)}, \qquad 1 < S < N.$$

例如 $N = 100, S = 90$：

$$f \geq \frac{100(89)}{90(99)} \approx 0.998878,$$

序列比例只能約 $0.1122\%$。若 $S > N$，此理想模型不可達；$S = N$ 在 $f = 1$ 且新增成本為零時可達。只要 $f < 1$，有限 $N > 1$ 才有嚴格的 $S_N < N$。

### 原書型反例

程式 baseline 為 100 s，其中乘法占 80 s、其他運算占 20 s。若目標是總時間 20 s，即 5 倍速度，則

$$20 = \frac{80}{s_{\text{mul}}} + 20.$$

# 單核到多核與 Amdahl 的完整推導 (續)

移項後 $80/s_{\text{mul}} = 0$，只有 $s_{\text{mul}} \to \infty$ 才趨近，因此有限加速不可能達到 5 倍。若只要求 2 倍，$T_{\text{new}} = 50$ s：

$$50 = \frac{80}{s_{\text{mul}}} + 20 \Rightarrow s_{\text{mul}} = 80/30 = 2.667.$$

這就是 Amdahl 的實作意義：先量「受改善部分占 baseline elapsed time 多少」，再問局部加速能否覆蓋其餘固定成本；不能把 operation count 或 source-line 比例直接當時間比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 製造與 AMD Opteron 案例

製造流程包含 wafer fabrication、測試、封裝與 system integration。Yield learning 與 design-for-test 會影響可用晶片數。效能 benchmark 只描述已配置的機器，不能單獨代表製造成本。

本章案例 AMD Opteron X4 是 2000 年代末的案例。閱讀時要保留年代，不能把它與今天的 core、cache、memory bandwidth 直接比較。

技術案例的用途是理解設計取捨。設計細節應回到當時的 ISA、製程、workload 與 server configuration。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# SPEC CPU 基準

SPEC CPU 的指標包含 single-thread 與 throughput 類。Single-thread 指標描述一個程式執行速度，throughput 指標描述指定並行配置下完成的 work。

Reference score 不是秒數。它是相對於 reference machine 的標準化結果，所以不能把兩個 ratio 直接相加成「總效能」。

最佳實務是保留每個 benchmark 的 ratio、硬體配置、compiler flags 與輸入。幾何平均可作摘要，但不取代個別結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## SPEC power 基準

Power benchmark 把 performance 與 active-idle energy 一起衡量。常見思維是指定工作負載下，每秒工作的總能源成本。

尖峰效能與 idle power 都會影響資料中心。若伺服器大部分時間低負載，idle 與 power-state transition 的成本會顯著。

Energy 與 power 要分清楚。$E = \int P(t)\,dt$。Power 的單位是 watt，energy 的單位是 joule。完成相同工作時比較 energy 才有意義。

不要用瓦數直接除以秒。先確認資料是平均功率還是累積能量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.8 謬誤與陷阱

第一個謬誤是局部加速比例等於整體加速比例。若該部分只占 baseline 的 10%，即使無限加速，總時間最多只降低 10%。

第二個陷阱是不管 workload。處理器在某個 benchmark 強，不代表在 database、compiler 或數值程式也強。

第三個陷阱是忽略利用率與成本。購買更多資源若不能提高完成工作數，可能只增加 energy 與管理成本。

三個可核對反例分別是：baseline 100 ms 中10 ms加速無限倍，仍需90 ms，speedup上限100/90；系統X在R1花1 ms、R2花10 ms，Y分別花2 ms、5 ms，所以優勝者隨workload改變；若單一link最多100 requests/s，增加CPU而不改link不能把持續完成率推過100。三例分別違反「局部等於整體」、「單一benchmark代表所有負載」及「資源數等於可交付吞吐」的假設。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 抽象與介面小結

抽象將複雜度分層，ISA 是最重要的硬軟體介面之一。效能來自 workload、ISA、compiler、microarchitecture、memory 與 I/O 的共同結果。

本章的四個檢查問題是：

1. 比較的是 latency 還是 throughput？
2. 指令數、CPI、frequency 哪一項改變？
3. 哪些資源是共享的，是否已飽和？
4. benchmark 與系統配置能否重現？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 歷史視角

電晶體整合提升可放在晶片上的功能，也讓設計從手工邏輯轉向 HDL 與合成。Pipeline、cache 與 out-of-order execution 逐步提高單核 ILP，但 power 與 complexity 設下限制。

2000 年代後，多核、hardware multithreading、SIMD 與 GPU 把平行度帶入主流。Compiler、runtime 與作業系統必須共同利用這些資源。

歷史比較要固定 ISA 與 workload，否則容易把世代、實作與應用變化混在一起。

電晶體與架構提供可用資源；只有 workload 的 critical path、指令工作量或資料搬移成本實際下降，才形成可量到的 application speedup。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 章末驗收

學生應能以一個程式例子寫出完整效能式，指出每個符號的單位，說明基準，並畫出 ISA 與微架構的位置。學生也應說明功率、溫度與 work parallelism 如何限制多核加速。

完成本章獨立練習題 PDF 後，以詳解 PDF 核對推導。程式例子只驗證本章的數字關係，不取代對假設與 workload 的討論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 參考資料

Patterson 與 Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，1.4–1.8，印刷頁 27–53（PDF 頁 54–80）。

完整書目、版本與未交付範圍見 章節導覽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study